Build Your Own RAG System from Scratch

Talk to Any Document with AI

Hands-on Workshop | Olamilekan M. Omotosho

Agenda

Here's what we will cover today

Why RAG Exists

Embeddings and
Semantic Search

Build the
Document Pipeline

Retrieve and
Generate Answers

What Is an LLM?

- Predicts the next token
- Strong at language tasks
- Weak at knowing your private documents
- Can hallucinate when unsure

The Hallucination Problem

~30%+ of incorrect responses

AI can mislead without correct data

Many users receive **confident answers** from AI systems, but without proper context or data, these responses can be misleading or entirely wrong.

(HalluHard benchmark, 2026)

What Is RAG?

Retrieval-Augmented Generation

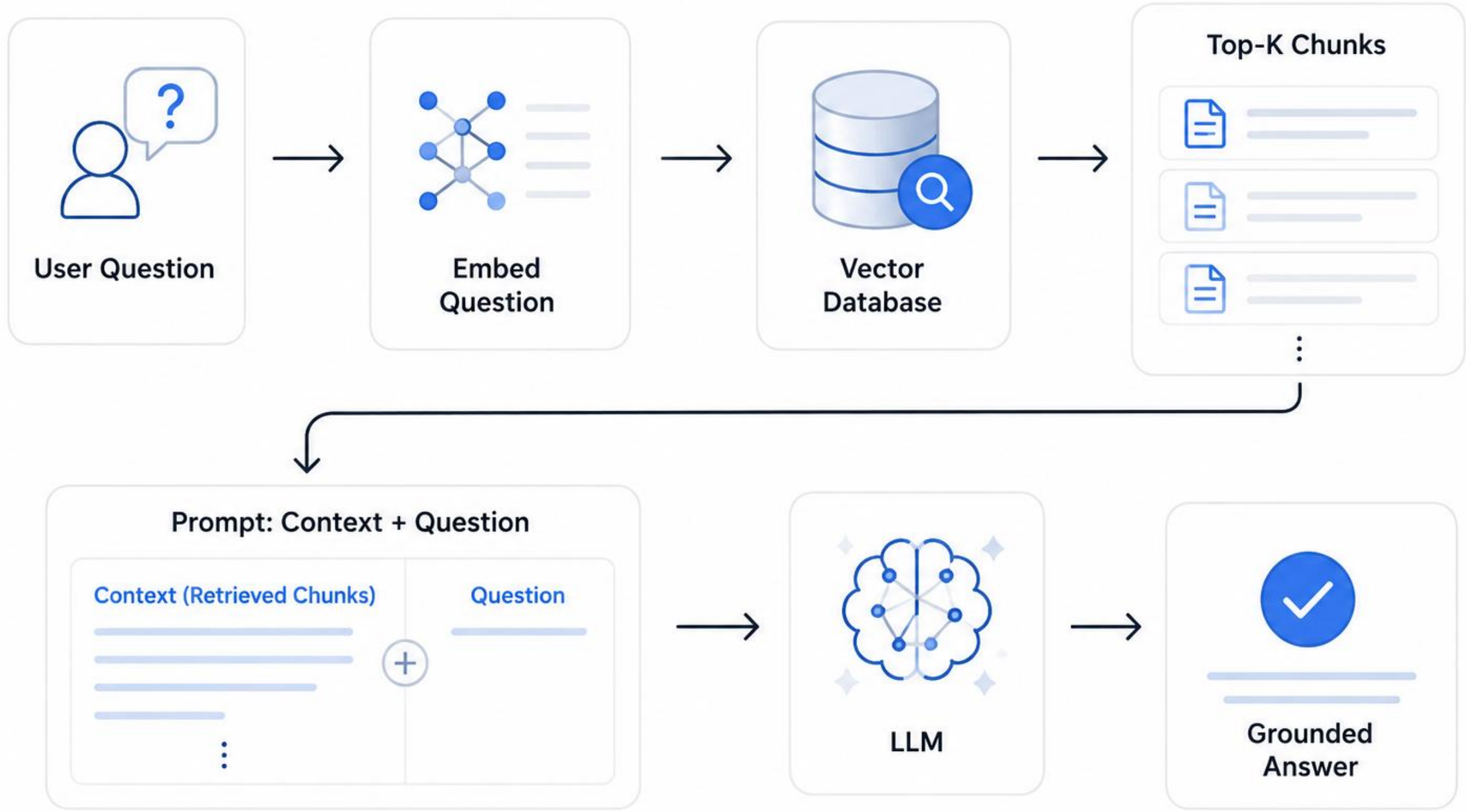
Retrieval-Augmented Generation (RAG) enhances AI models by providing relevant context, ensuring more accurate and grounded answers to user queries based on precise data retrieval.

Give the model the right context before it answers.

Real-World Applications

- FAQ
- University Course Assistant
- Enterprise Knowledge Assistant
- Research & Literature Q&A
- ChatGPT document tools

RAG Architecture?

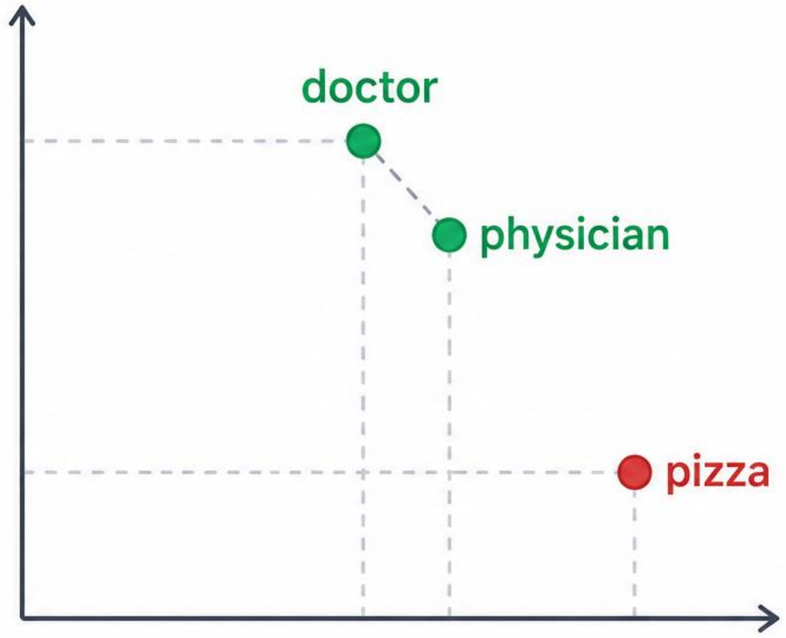


What Are Embeddings?

Text → numbers that capture meaning

Similar meaning → close vectors

doctor + **physician** > **doctor** + **pizza**



Points closer together mean more similar meaning.

Why Chunk Documents?

- LLMs have context limits
- Smaller chunks improve retrieval precision
- Overlap preserves meaning at boundaries

Vector Database

- Stores document meanings as vectors (embeddings)
- Handles millions of vectors efficiently
- Finds similar content based on meaning, not keywords
- Today: Using **ChromaDB** running locally on your laptop

"Can I travel with my dog?" retrieves **pet policy**, **animal transport rules**, and **cabin restrictions**, even if the word **dog** never appears.

Demo App Preview

Workshop Stack Overview

Python

Core programming language for building the RAG system.

LangChain

Framework facilitating the orchestration of various pipelines.

Sentence Transformers

Enables local embeddings for efficient semantic search operations.

ChromaDB

Vector database for storing and retrieving embeddings efficiently.

Gemini API

Provides answer generation capabilities to enhance user interactions.

Let's Build

Open the **notebooks/01_rag_workshop.ipynb** file to begin your hands-on experience. Focus on **Section 2: Embeddings**, where you will learn to create effective document embeddings for your RAG system.

The Retrieval Step

Understanding how to extract context

Question as Vector

The user's question is transformed into a vector, enabling the system to effectively represent it in a mathematical format for processing.

Top-K Chunk Retrieval

By retrieving the top-k relevant chunks, the system ensures that only the most pertinent information is passed to the LLM for generating accurate responses.

Prompt Construction

Essential Steps for Effective Responses

System Prompt

The system prompt sets the context, guiding the model's behavior and tone. It ensures the generated responses align with user expectations and desired outputs.

Retrieved Context

Retrieved context provides the model with specific data relevant to the user's inquiry, enhancing response accuracy and groundedness in the provided information.

User Question

The user question captures the inquiry's essence, allowing the model to focus on delivering precise answers based on the combined context and prompt guidelines.

RAG Comparison

Without RAG

Relies on training data for guesses and assumptions.

With RAG

Utilizes your documents to provide accurate answers.

Grounded Answers

Answers include sources for better credibility and trust.

Local App Demo

```
cd rag-workshop
streamlit run app.py
```

Use Cases by Audience

Lecturers

Build course assistants tailored to specific student needs.

Researchers

Simplify literature reviews and policy inquiries with ease.

Industry

Prototype internal FAQs to enhance team knowledge sharing.

Students

Access study materials and resources through AI assistance.

Administrators

Streamline documentation and policy retrieval processes efficiently.

Developers

Integrate RAG systems into existing applications for enhanced functionality.

Make It Yours

Customization

Add your own .txt or .pdf files for personalization.

Vector Store

Rebuild the vector store to suit your document needs.

Testing

Test real questions to validate your RAG system's functionality.

Application

Use RAG in practical scenarios to enhance engagement and learning.

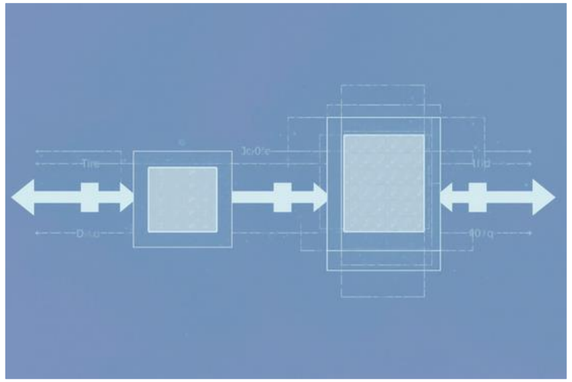
Integration

Seamlessly integrate custom data into your existing workflows.

What You Built Today: Key Components

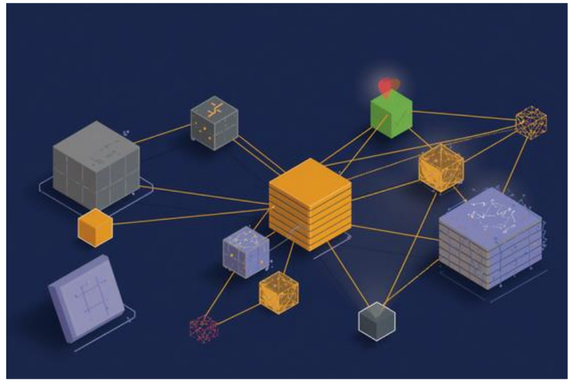
Loader

The document loader manages incoming data efficiently.



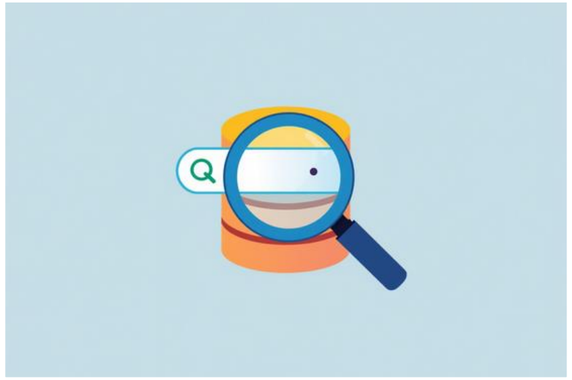
Chunking

Chunking enhances data retrieval precision during querying.



Storage

The local vector store keeps embeddings organized effectively.



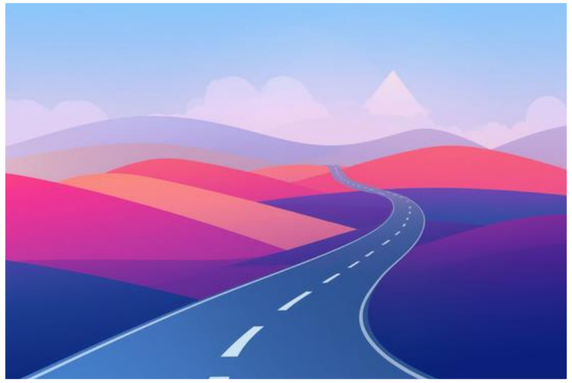
Retrieval

The retrieval function fetches relevant data efficiently.

Answers

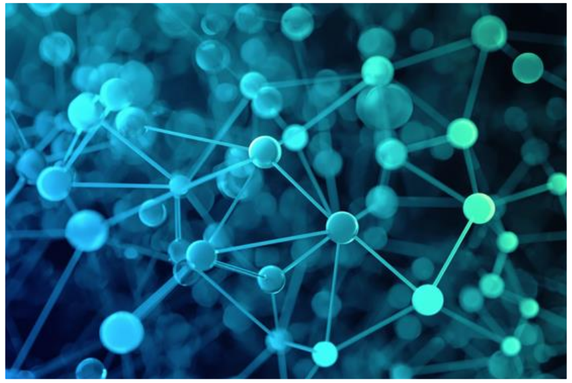
The RAG function generates contextually grounded responses.

Learning Roadmap for Your RAG Journey



Retrieval

Enhancing techniques for better data retrieval outcomes.



Agents

Development of adaptive AI agents for dynamic tasks.

Fine-tuning

Tailoring models for specific tasks and improved outcomes.

Multimodal

Integration of various data types for comprehensive understanding.

Engineering

Building robust systems for real-world applications and scalability.

Make It Yours

- LangChain docs: https://python.langchain.com/docs/
- Chroma docs: https://docs.trychroma.com/
- Gemini API: https://ai.google.dev/
- Hugging Face NLP course: https://huggingface.co/learn/nlp-course/

Thank You for Attending!

Olamilekan M. Omotosho

Build Your Own RAG System from Scratch